

Einführung in die Programmierung (WIN+BIT)

Grundlagen Java

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger

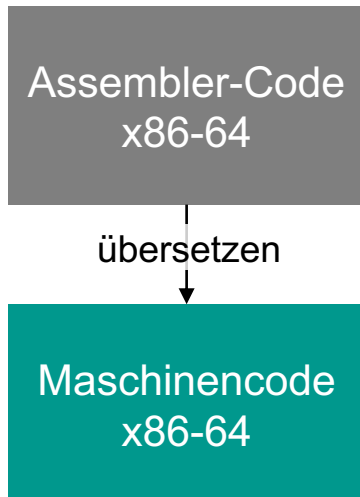


Inhaltliche Ziele

“ Die Studierenden kennen alle **wesentlichen Konzepte objektorientierter Programmierung** und sind insbesondere **vertraut mit den Konstrukten der Programmiersprache Java**.
Die Studierenden haben in betreuten Laborübungen gelernt, selbständig geeignete **Programmierungsmethoden zur Lösung praktischer Probleme** anzuwenden.
Damit haben sie die Schlüsselkompetenzen **Methodenkompetenz** und **Selbstkompetenz** gestärkt.

Aus dem Modulhandbuch.

Programmieren in Assembler



```
ASSUME  CS:CODE, DS:DATA

DATA    SEGMENT
Meldung db  "Hello World!"
        db  13, 10
        db  "$"
DATA    ENDS

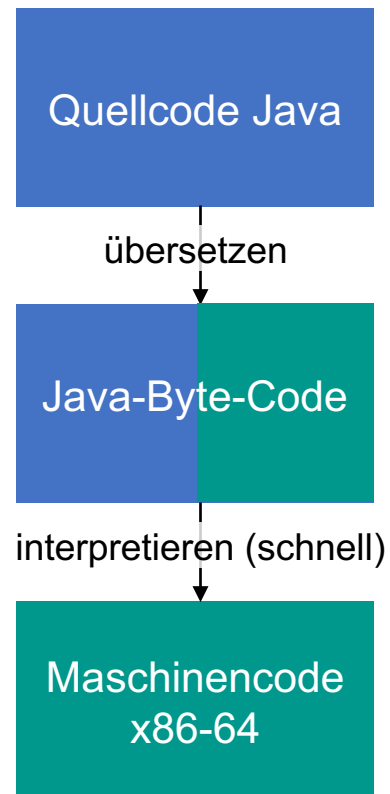
CODE    SEGMENT
Anfang:
        mov ax, DATA
        mov ds, ax
        mov dx, OFFSET Meldung

        mov ah, 09h
        int 21h
        mov ax, 4C00h
        int 21h
CODE    ENDS

END     Anfang
```

Programmieren in Java

```
System.out.println("Hello Students!")
```



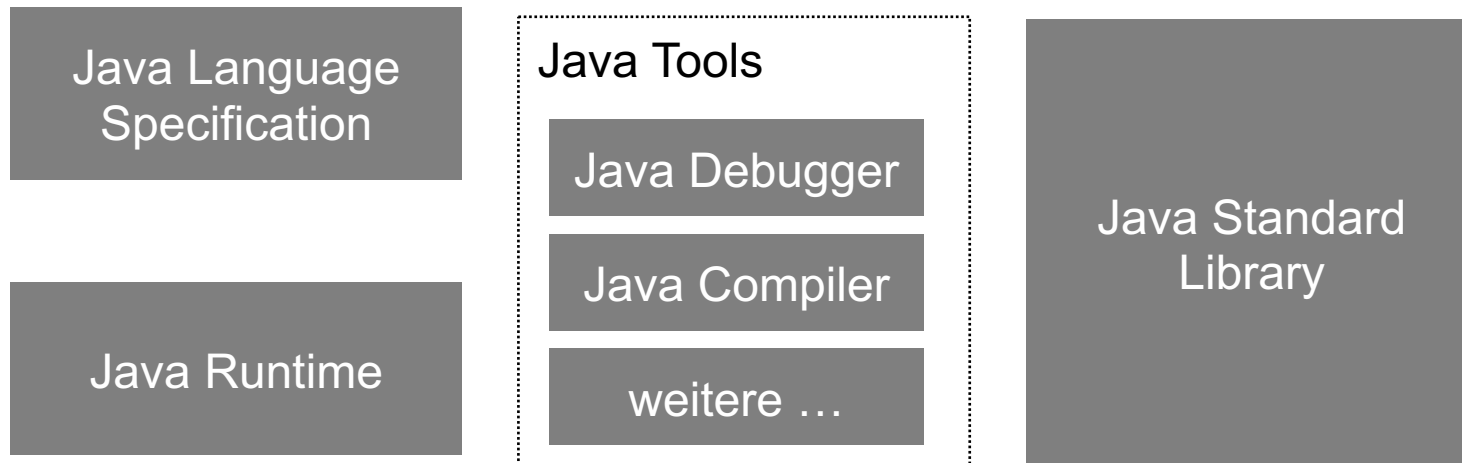
Programmiersprachen

- Heute fast nur noch **höhere** Programmiersprachen
 - **Rechnerunabhängig**
 - **Komfortablere** Programmierung durch **abstrakte** Konzepte, u.a.
 - Prozeduren, Subroutinen, Methoden
 - Module
 - Klassen, Schnittstellen, Vererbung
 - **Weniger effizient** als Assembler
 - Wesentlich schneller und **billiger in der Wartung** (Programme korrigieren, ändern, erweitern)
 - Es gibt **sehr viele** höhere Programmiersprachen
https://en.wikipedia.org/wiki/Category:High-level_programming_languages

Die Programmiersprache Java und das JDK, was gehört dazu?

Java Development Kit

- Java Language Specification
- Java Tools
- Java Runtime (Java Virtual Machine)
- Java Standardbibliothek (Standard Library)



Java Language Specification

Java Language
Specification

Java Tools

Java Standard
Library

Java Runtime

- Syntax (=formale Schreibweise) der Sprache
- Wird vom Compiler überprüft
- aktuelle Version [Java SE 17](#)

4.2. Primitive Types and Values

A primitive type is predefined by the Java programming language and named by its reserved keyword (§3.9):

```
PrimitiveType:  
  {Annotation} NumericType  
  {Annotation} boolean  
  
NumericType:  
  IntegralType  
  FloatingPointType  
  
IntegralType:  
  (one of)  
  byte short int long char  
  
FloatingPointType:  
  (one of)  
  float double
```

Java Tools

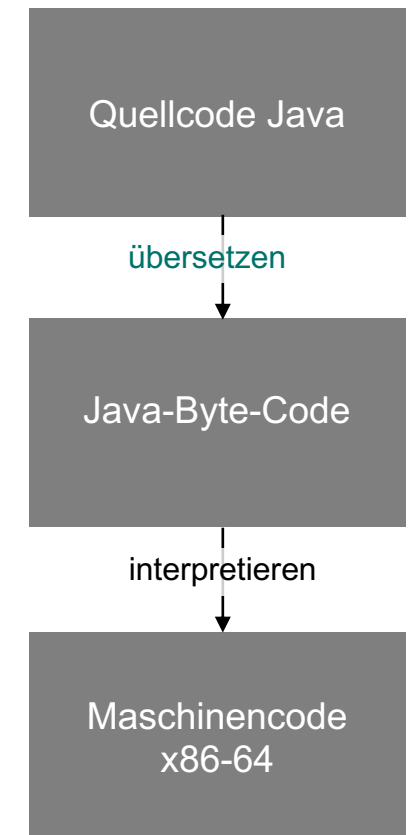
Java Language
Specification

Java Tools

Java Standard
Library

Java Runtime

- Java-Compiler: **Übersetzt** Programm-Code in Java-Bytecode
- Java-Debugger: Ermöglicht das schrittweise Nachvollziehen einer Programmausführung zum Zweck der Fehlerfindung (Bug 🐛)
- Java-Dokumentationswerkzeug: Erstellt HTML-Dokumentation für den Code
- Java-Archiver: Packt kompilierten Byte-Code zu einer in anderen Projekten verwendbaren Programmbibliothek zusammen (JAR-Datei)



Java Runtime

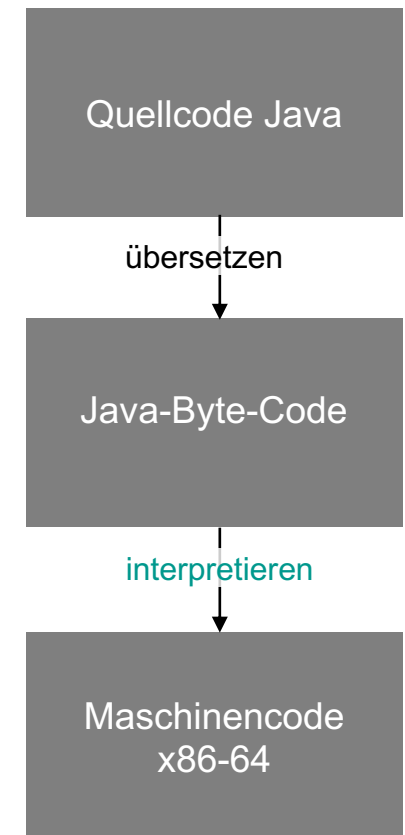
Java Language
Specification

Java Tools

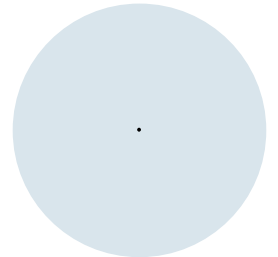
Java Standard
Library

Java Runtime

- Java Virtual Machine (JVM)
- Ausführungsumgebung für den Java-Byte-Code
- Byte-Code wird **interpretiert** und in Form von nativem Maschinen-Code ausgeführt
- „Just in time“-Compilation (JIT): Optimierung der JVM: Häufig verwendete Java-Byte-Code-Folgen werden während der Ausführung zu effizienterem Maschinen-Code compiliert



Java-Programme



Erstellen von Java-Programmen

- Das einfachste Java-Programm besteht aus einer einzelnen Klasse, z.B. `MyClass`
- Diese Klasse muss in einer Datei des Namens `<Klassenname>.java` definiert sein.
Hier: `MyClass.java`
- Der Java-Compiler (`javac`) übersetzt diese Klasse in **Bytecode** und speichert das Ergebnis in der Datei `MyClass.class`
- Die virtuelle Maschine (**JVM**) interpretiert diesen Bytecode.

Organisation von Source-Code

- Eine **Klasse** = eine **Datei** (Endung .java)
- Klassen **kapseln** eine gewisse Funktionalität
 - "abkapseln" von anderen Klassen
 - "bündeln"
- In den ersten Vorlesungen und Übungen:
 - Wir schreiben **eine** Klasse
 - diese hat eine **main-Methode** mit dem Code, den wir ausführen
 - Wir rufen hier auch Funktionen aus anderen Klassen des JDKs auf, z.B. Math oder Scanner
- später: Klassen als Grundlage für *objekt-orientiertes Design*

Packages

- Packages (Pakete) sind **Sammlungen von Klassen**
- Werden abgebildet **im Dateisystem als (Unter-)Verzeichnisse**
- Alles in src direkt ist im sogenannten **Default-Package** (ohne Namen)
- Alle weiteren Unterverzeichnisse entsprechen **benannten Packages**

Java Standard-Bibliothek

Java Language
Specification

Java Tools

Java Standard
Library

Java Runtime

- auch Java-API genannt („Application Programming Interface“)
- [Version 21 API Specification](#)
- In jeder Java-Runtime verfügbare Standardfunktionalität
- Sammlung verschiedener APIs für
 - Eingabe/Ausgabe (Tastatur, Dateisystem, Netzwerk, Datenbanken)
 - Mathematische Funktionen
 - Datenstrukturen (Mengen, Listen, Bäume, Maps)
 - Graphische Benutzeroberflächen
 - u.v.m.

Beispiel Java-API

```
System.out.println("Hallo!"); // gibt Hallo aus
```

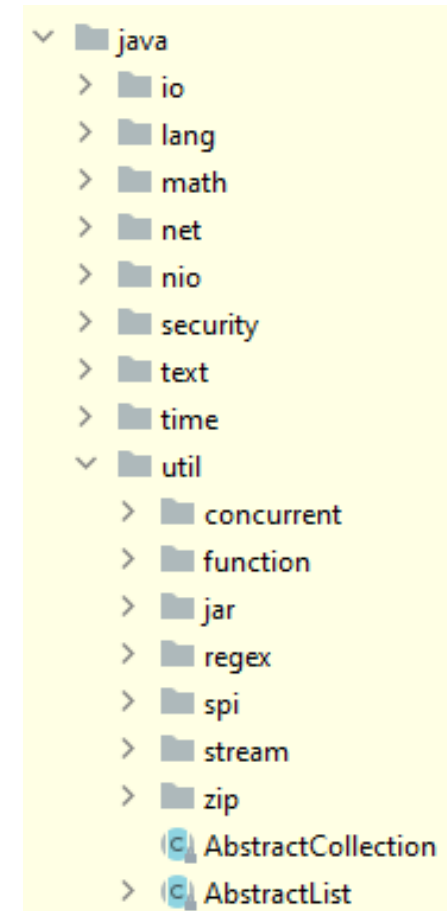
<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/System.html>

```
Math.log10(1000.0); // ergibt 3
```

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Math.html>

Java-Packages

- in der Java Runtime enthalten
- beginnen mit `java`
- Package `java.lang` für die wichtigsten Sprachfunktionen, z.B. `java.lang.System` für `System.out.println` und `java.lang.Math` für mathematische Berechnungen
- Package `java.io` und `java.nio` für nützliche Klassen zur Datei-Eingabe und -Ausgabe (später mehr)
- Package `java.util` für nützliche Klassen zum
 - Einlesen von Daten von der Konsole (später mehr) sowie
 - Datenstrukturen (→ SWEN1)



viele mehr...

Verwendung der Java-API in Übungen und Klausur

Wichtig: Lesen Sie die Aufgabenstellung genau!

Wenn es nicht anders angegeben ist, dürfen Sie für die Bearbeitung von Aufgaben keine Java-API-Funktionen verwenden, sondern müssen die Funktionalität selbst programmieren.

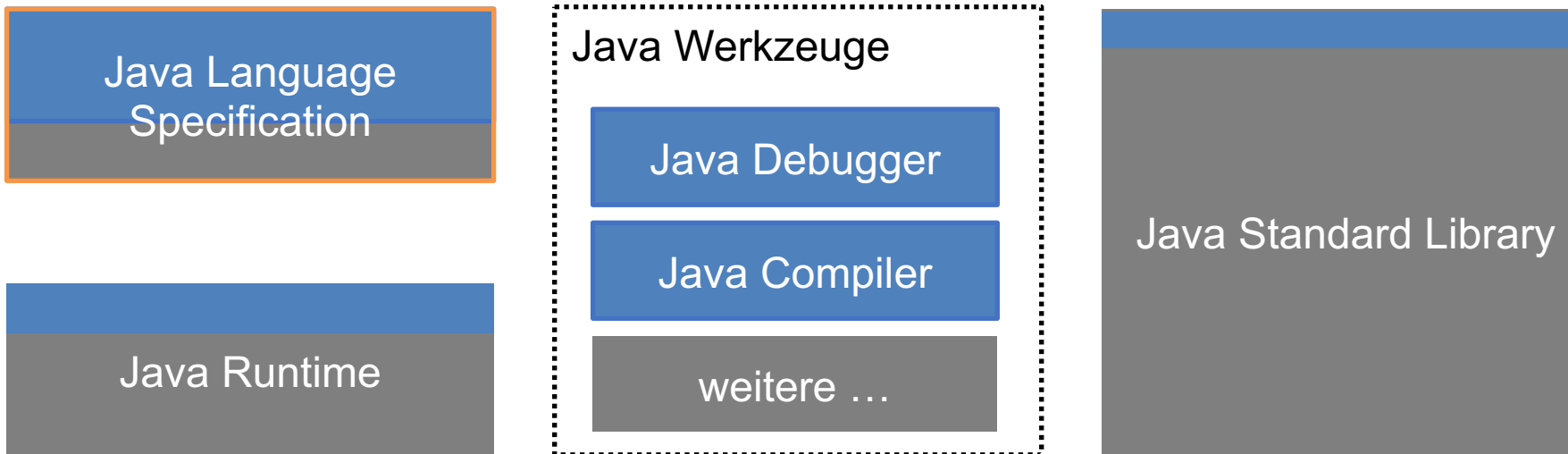
Beispiel: „Vervollständigen Sie den Programmcode, so dass in c das Minimum von a und b steht

```
int a, b, c;  
// a und b werden mit Werten belegt  
// Ihr Code hier
```

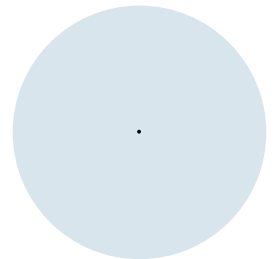
Hier darf man nicht einfach `c = Math.min(a,b);` schreiben!

Technisches Wissen am Ende der Veranstaltung

Fokus auf Sprache und Sprachkonstrukte



Einführung in Versionskontrolle mit git





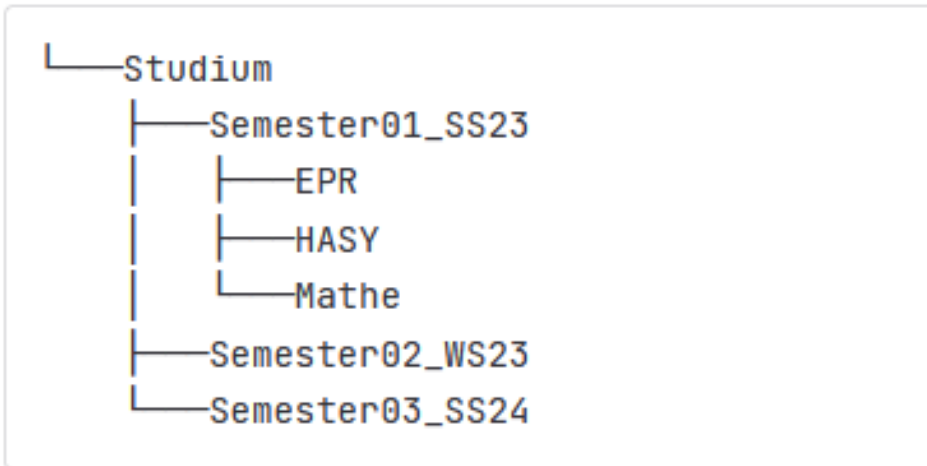
Dateien und Verzeichnisse

- Machen Sie sich mit Ihrem Dateisystem vertraut (Wo lege ich Dateien ab? Wie kann ich Dateien



Tipp 9: Dateisystem strukturieren

- Machen Sie sich mit Ihrem Dateisystem vertraut (Wo lege ich Dateien ab? Wie kann ich Dateien kopieren, verschieben, löschen, umbenennen? Wie erstelle ich ein neues Verzeichnis/Ordner?)
- Wählen Sie einen Haupt-Ordner, unter dem Sie alles zu Ihrem Studium ablegen, z.B.



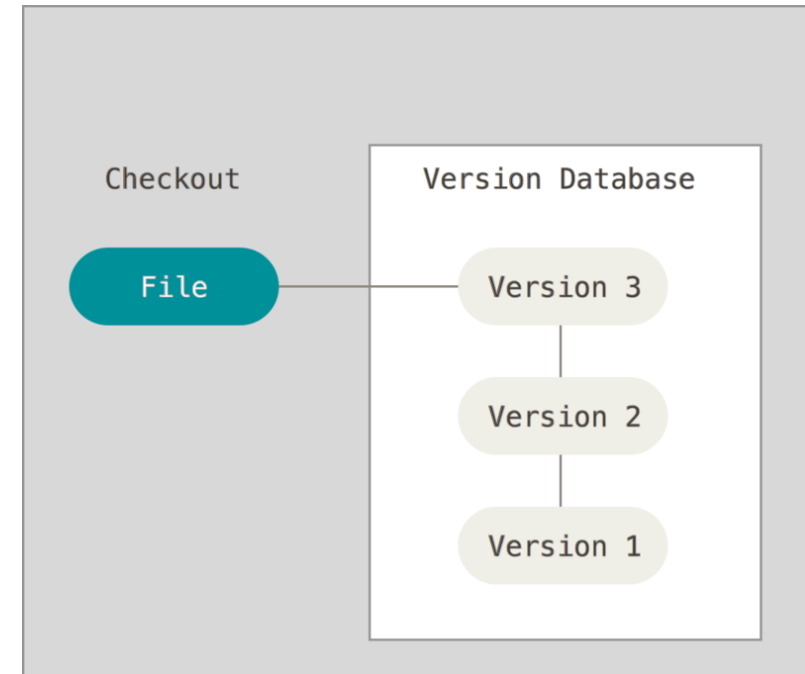
- Sichern Sie Ihre Daten regelmäßig, z.B. über die Nextcloud-Instanz von [bwSync&Share](#) (50GB Speicher, [Anleitung](#))

Probleme

- Mein Programm hat vorher fast funktioniert, aber jetzt geht es nicht mehr.
- Ich möchte mein Programm abgeben
- Ich möchte Programme herunterladen

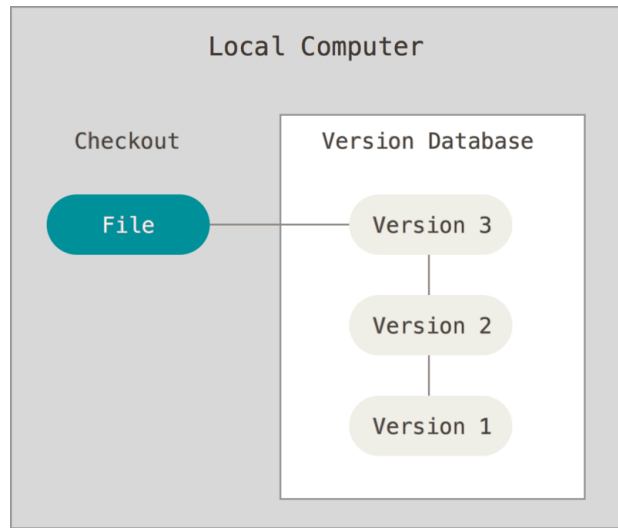
Was ist Versionsverwaltung?

- Speichern verschiedener Versionen einer Datei
- alte Version (auch gelöschte Datei) kann jederzeit wiederhergestellt werden
- Version Database nennt man auch *Repository*
 - Üblicherweise enthält diese mehrere Dateien und (Unter-)Verzeichnisse
- Das Holen einer Version aus dem Repository nennt man auch *Checkout*
- Das Erstellen einer neuen Version nennt man auch *Commit*



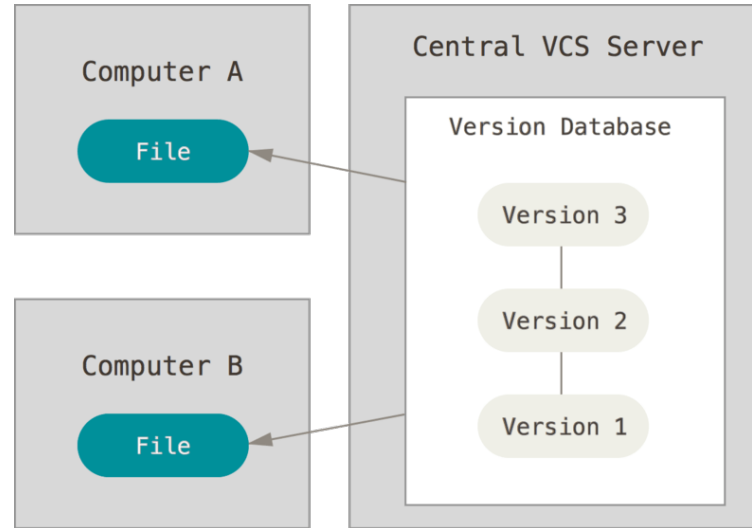
Welche Arten von Versionsverwaltung gibt es?

Zugriff auf mehrere Versionen einer Datei von einem Computer aus

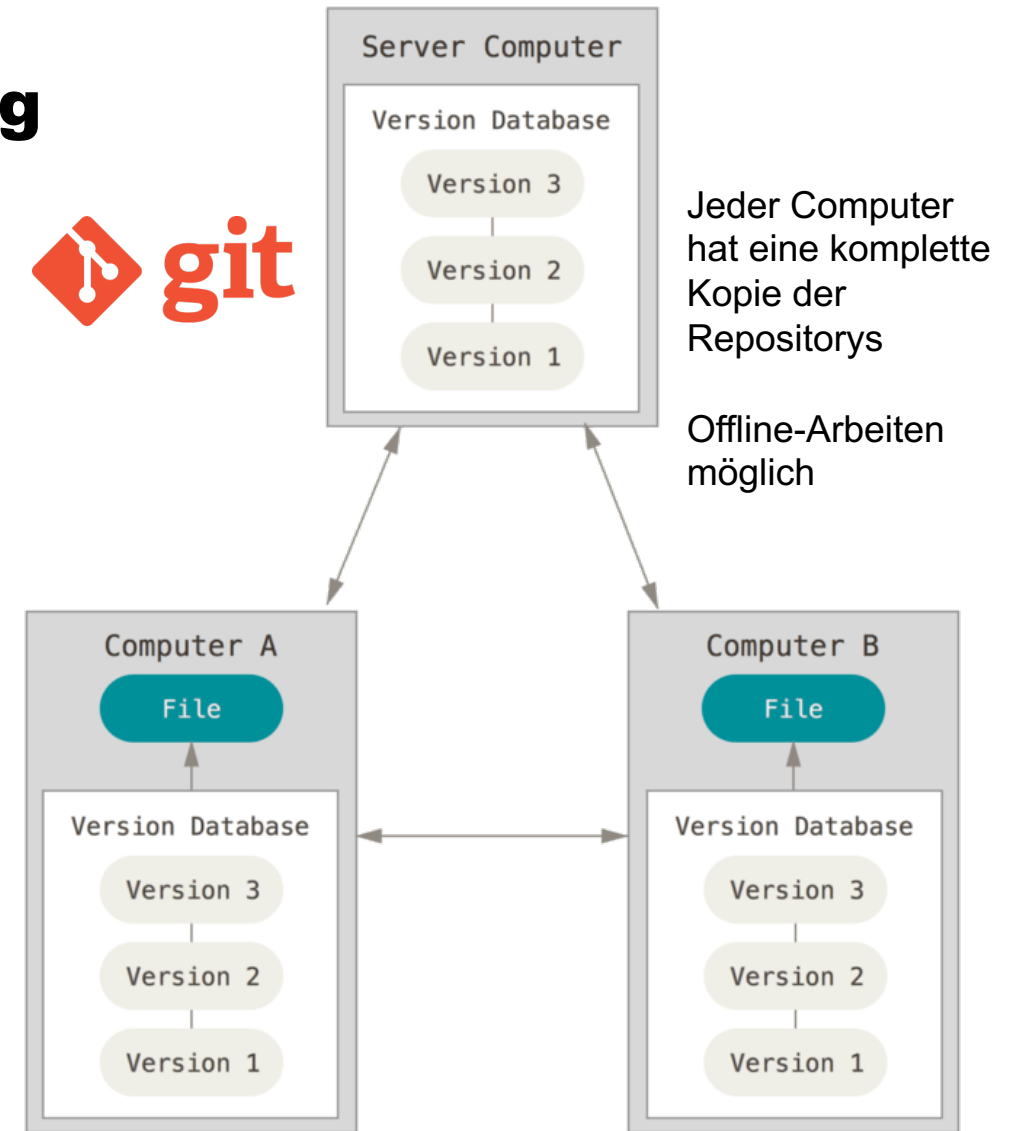


Lokale Versionsverwaltung

Zugriff auf mehrere Versionen einer Datei
Datei und Versionen liegen auf zentralem Server



Zentrale Versionsverwaltung

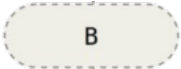


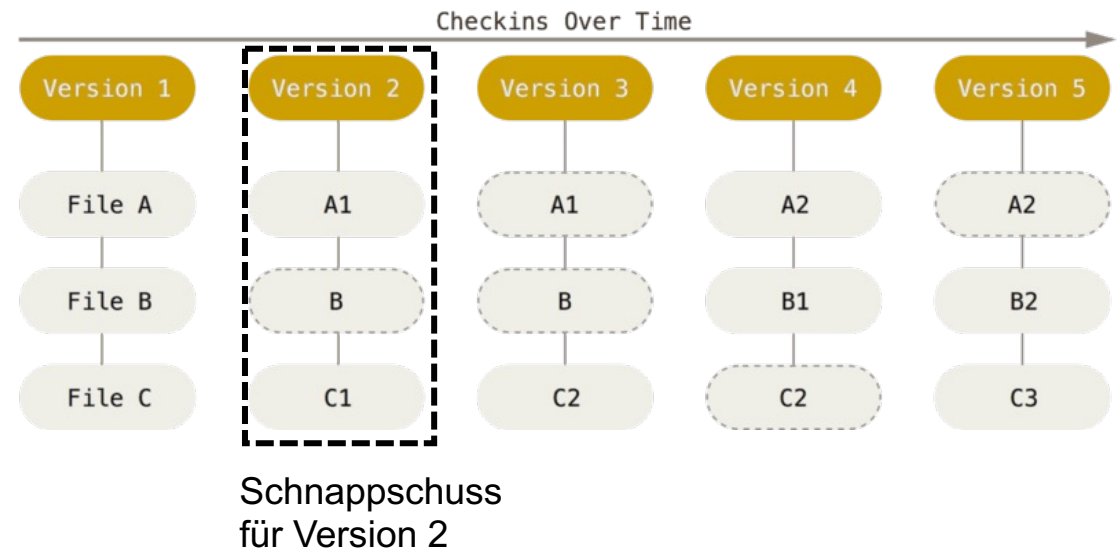
Dezentrale Versionsverwaltung

Jeder Computer hat eine komplette Kopie der Repositories

Offline-Arbeiten möglich

Wie speichert Git Versionen?

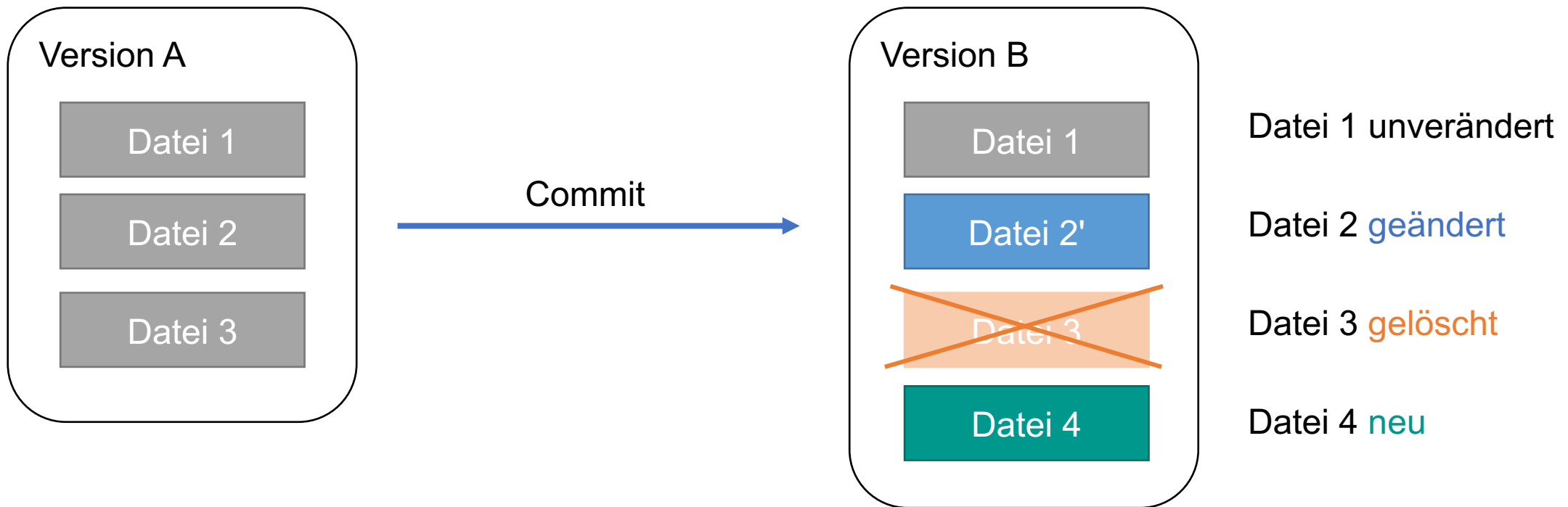
- Git speichert mit jedem *Commit* einen neuen Schnappschuss aller Dateien im Repository
- Um Speicherplatz zu sparen, werden unveränderte Dateien nur als Referenz gespeichert
 -  ist Referenz auf unveränderte Datei B



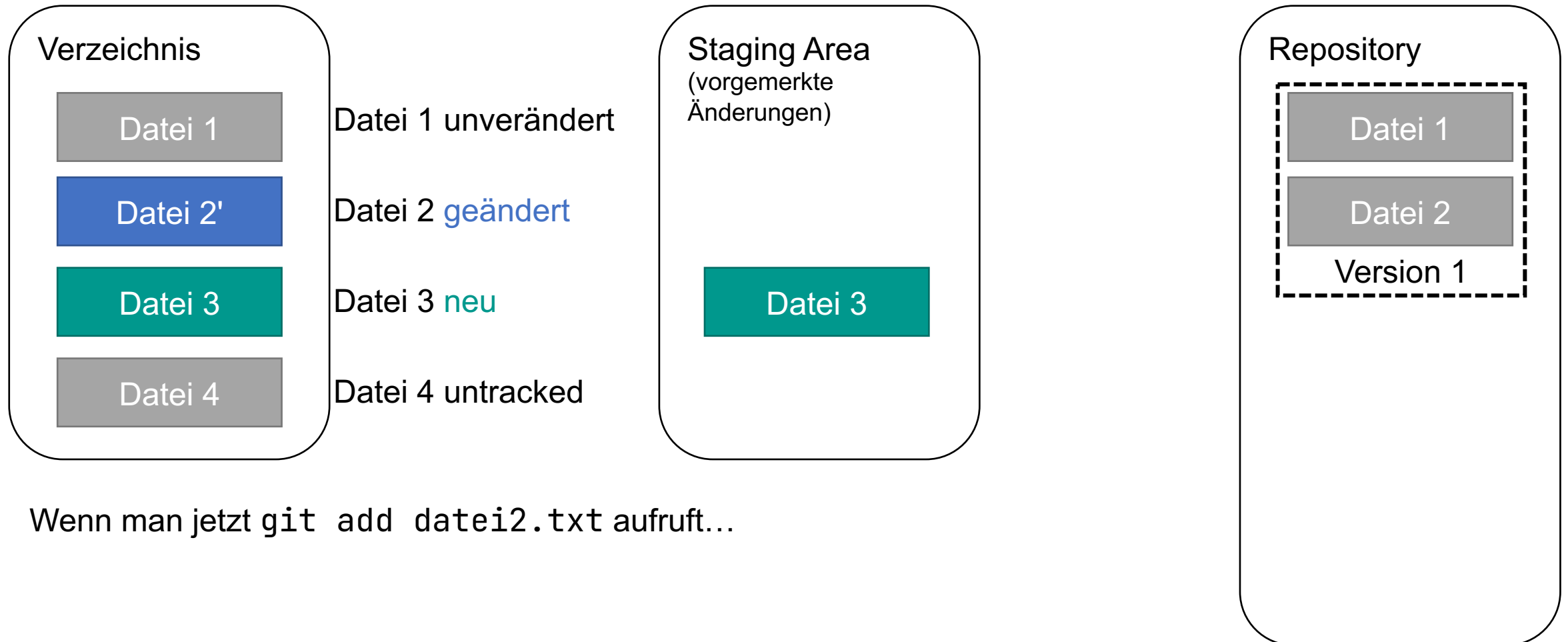
Warum kann man nicht einfach Dropbox nehmen?

- Dienste wie Dropbox speichern Änderungen pro Datei
 - Wenn jemand z.B. in fünf Dateien eine neue Kopfzeile einfügt, ist das in Git **ein** Commit, den man leicht rückgängig machen kann
 - In Dropbox sind das **fünf** Änderungen, die nicht zusammenhängen und zu (leicht) unterschiedlichen Zeitpunkten passiert sind
- Git-Commits erhalten eine kurze Beschreibung der Änderung
- Dropbox macht automatisch sofort einen Upload aller Änderungen im Hintergrund
 - Schlecht, wenn man etwas Neues ausprobiert, was noch nicht fertig ist, aber an alle anderen verteilt wird

Was kann ein Commit alles beinhalten?

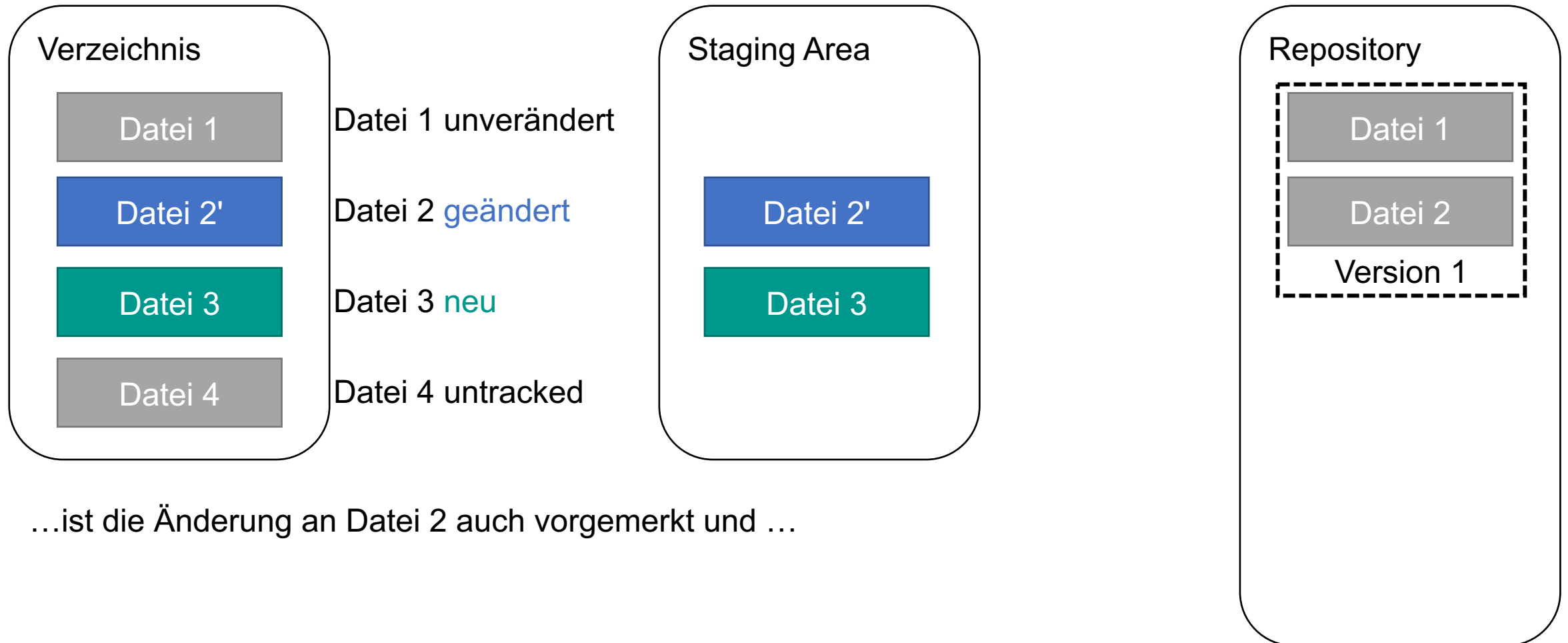


Beispiel, aktueller Zustand

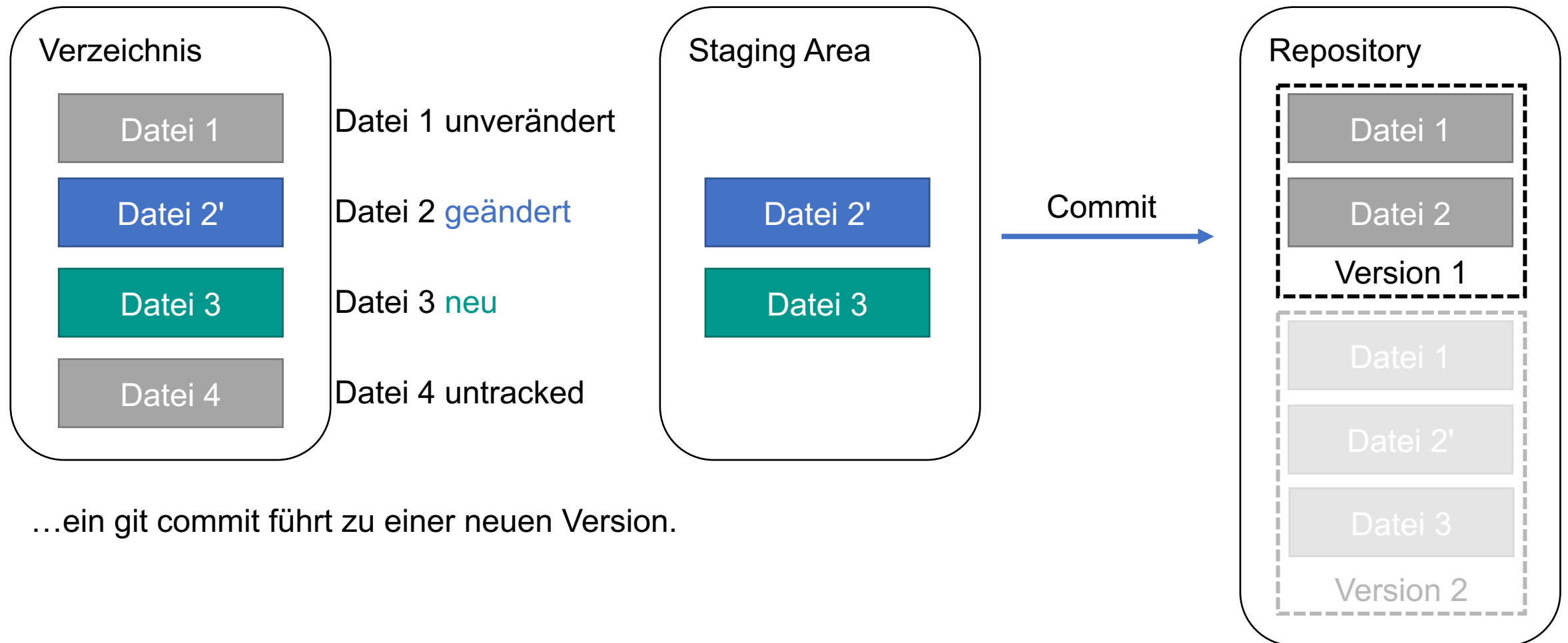


Wenn man jetzt `git add datei2.txt` aufruft...

Beispiel, nach git add datei2.txt

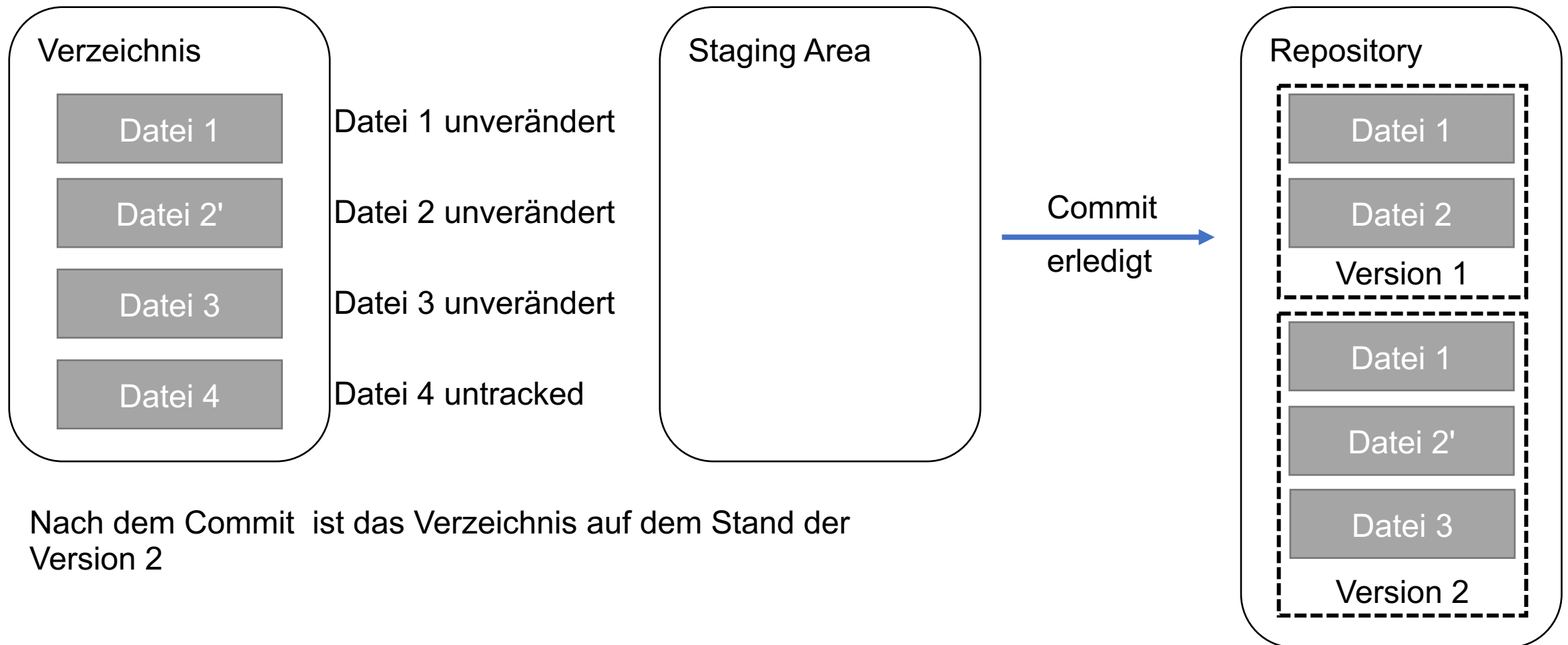


Beispiel, vor git commit



...ein git commit führt zu einer neuen Version.

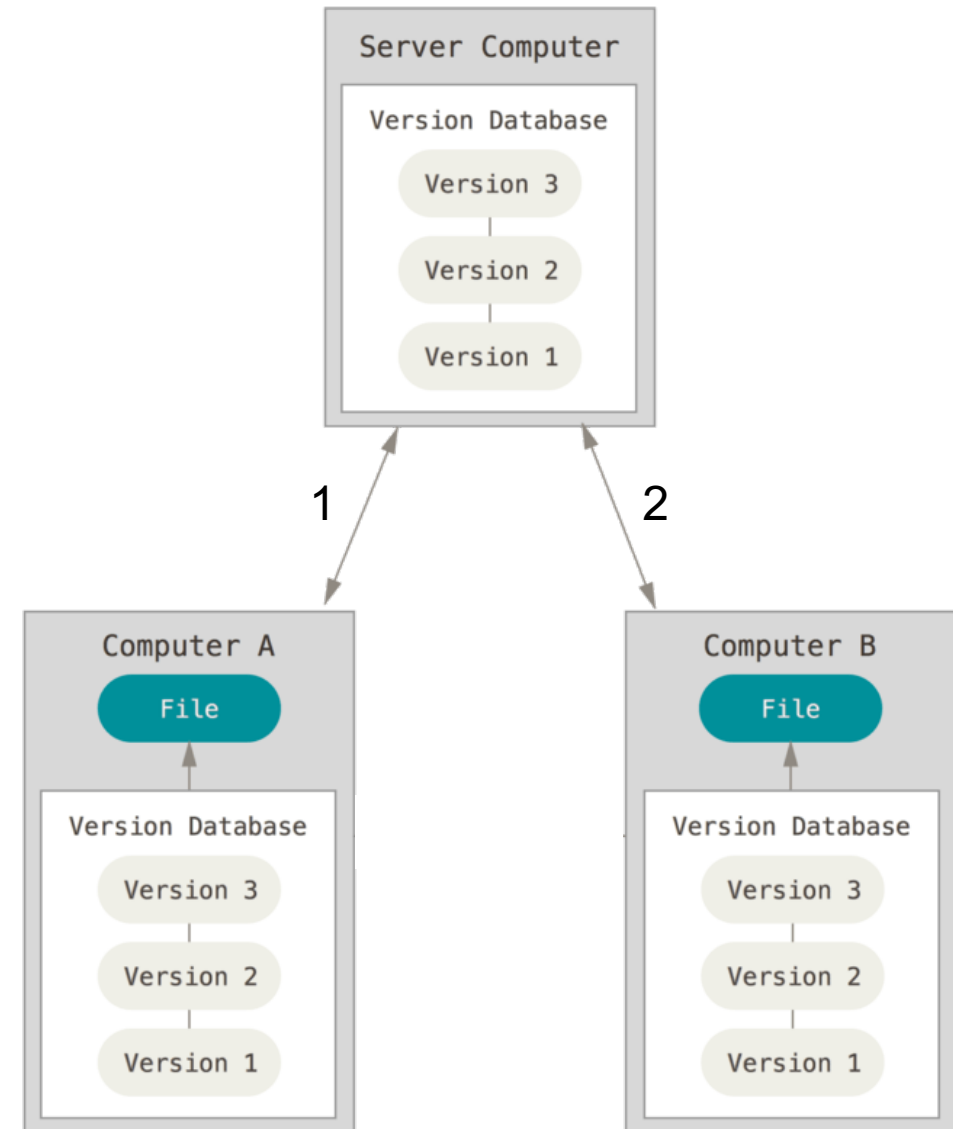
Beispiel, nach git commit



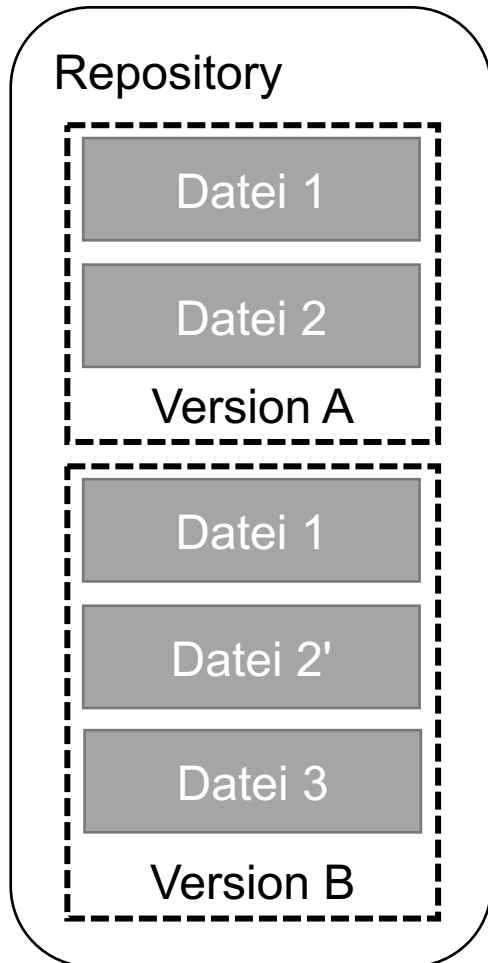
Nach dem Commit ist das Verzeichnis auf dem Stand der Version 2

Arbeiten mit lokalem und entfernten Repository

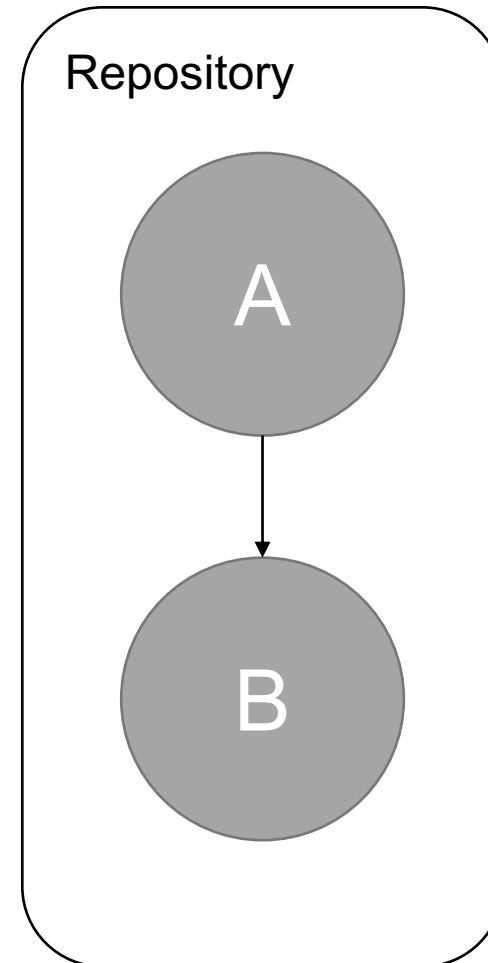
- Um Änderungen am Repository von Computer A an weitere Computer weiterzugeben, synchronisiert der Computer A erst sein Repository mit dem Server
- Anschließend kann Computer B sein Repository mit dem Server synchronisieren und erhält damit die Änderungen



Ab jetzt: Darstellung der Commits

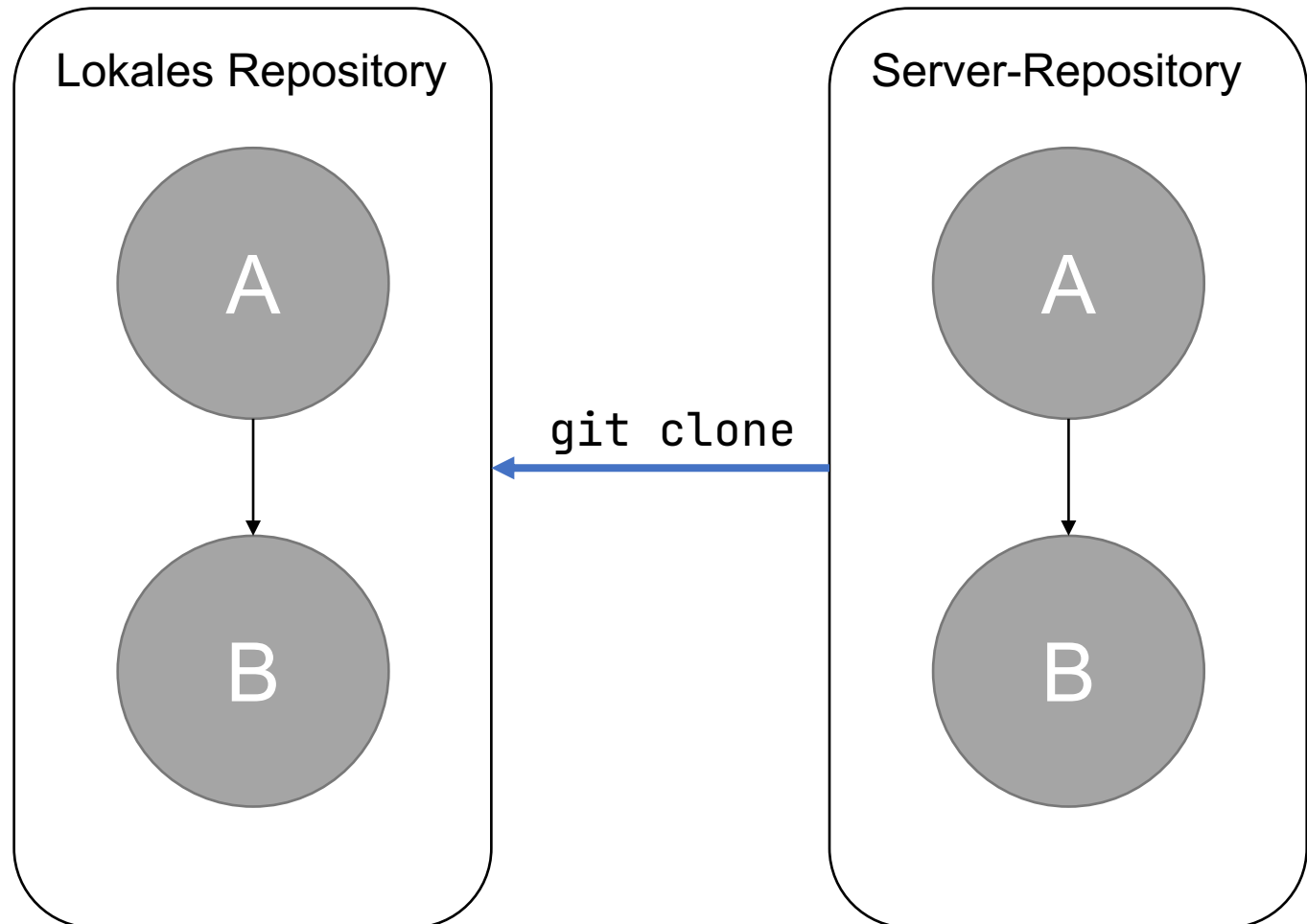


stellen wir vereinfacht dar als

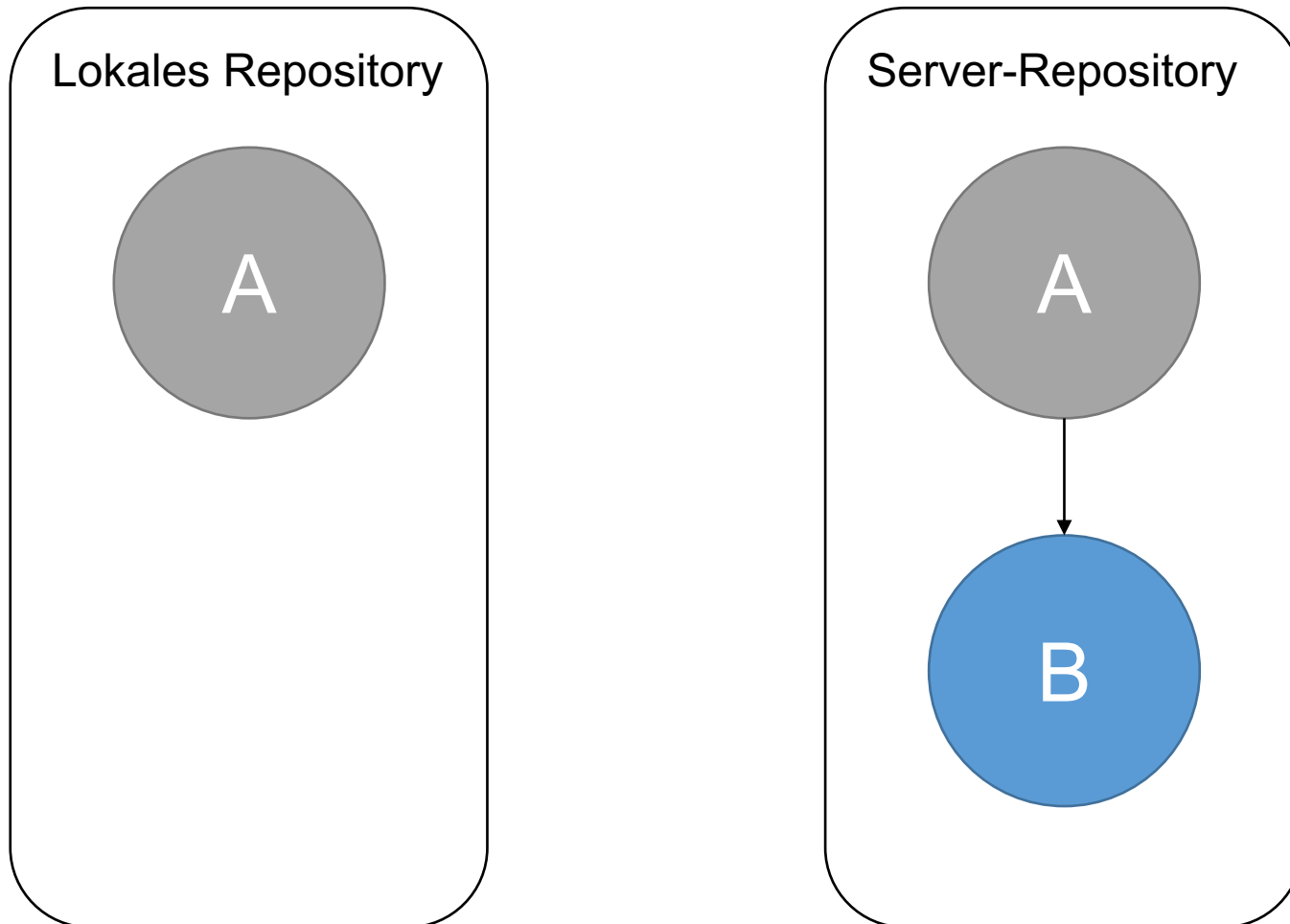


Initialer Download des Server-Repositorys

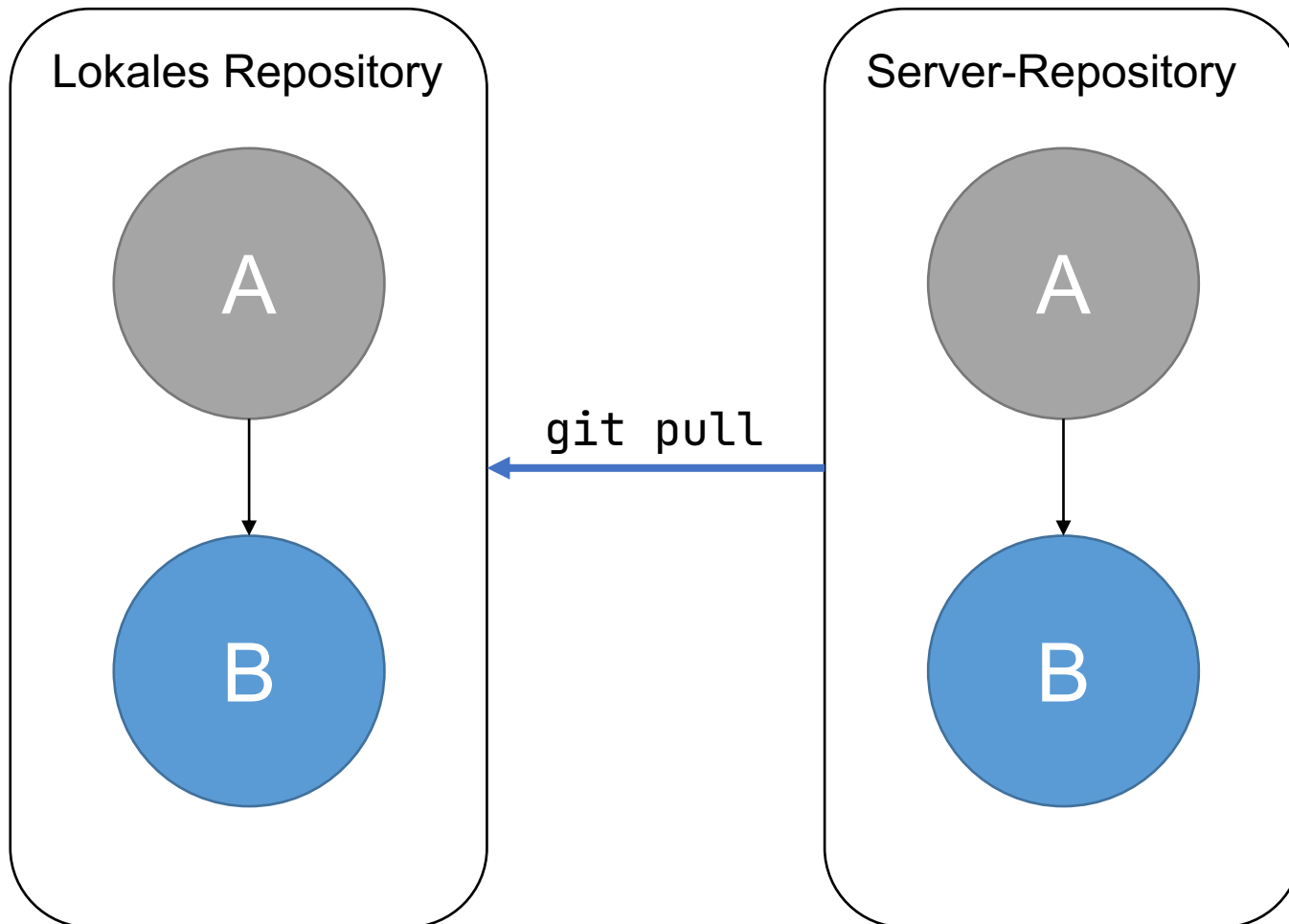
- nennt man "Klonen" (`git clone`)
- Üblicherweise klonst man ein vorhandenes Repository, z.B. von GitHub oder GitLab, um dann lokal daran zu arbeiten.
- Es geht auch umgekehrt: `git init` in einem Verzeichnis erzeugt ein lokales Repository, welches man später mit einem Server synchronisieren kann



Synchronisierung Server zu lokal

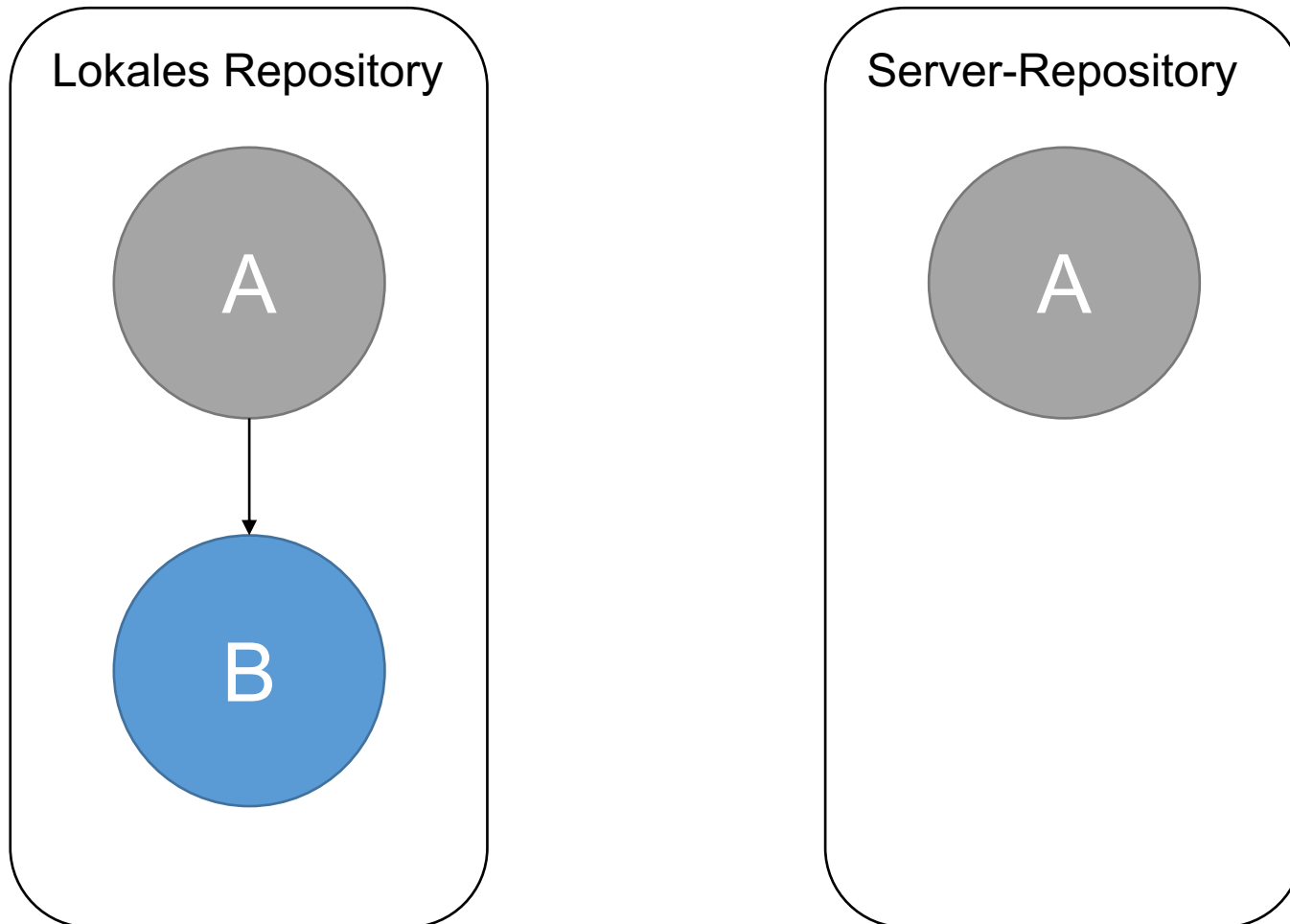


Synchronisierung Server zu lokal

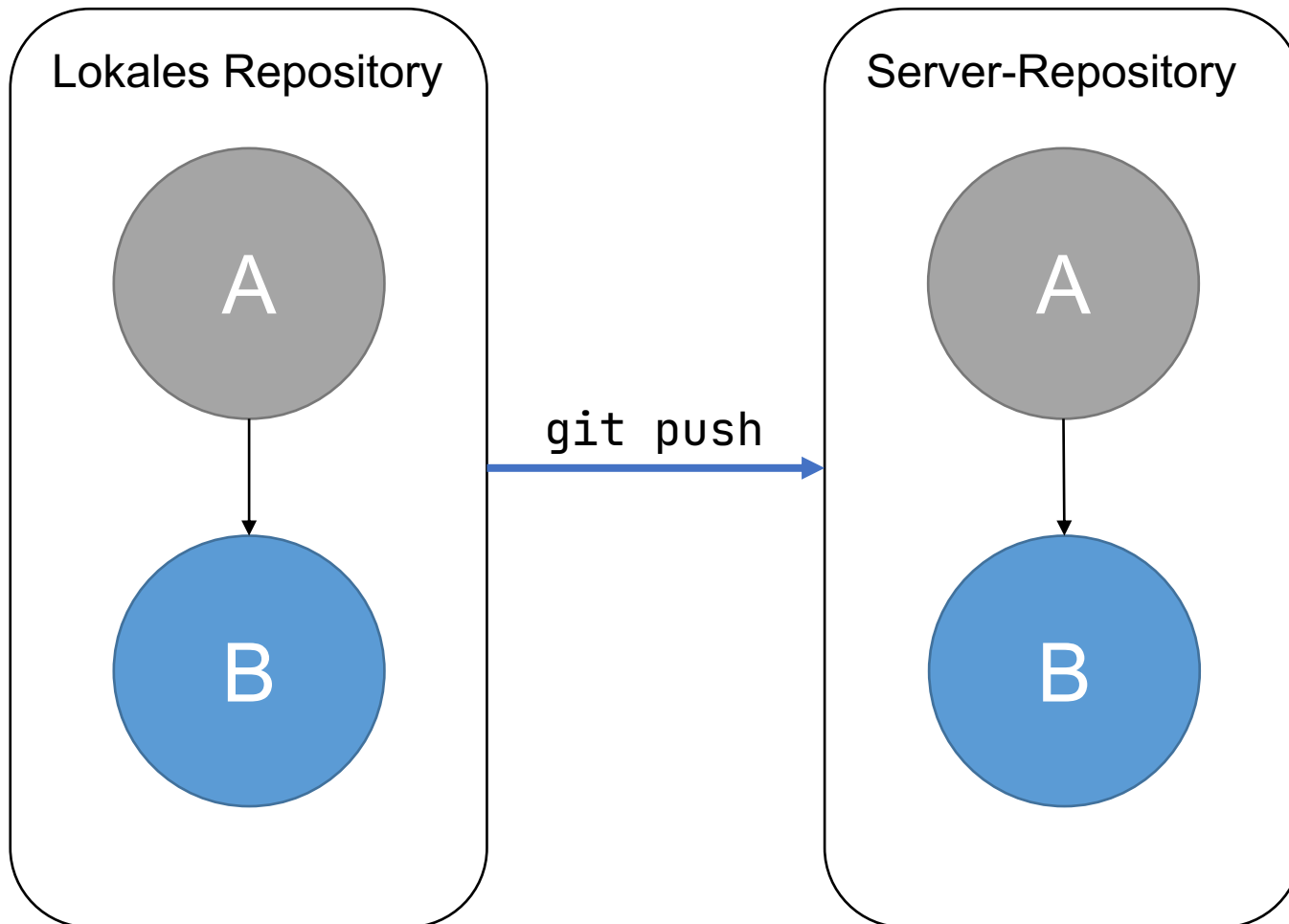


- `git pull` fügt der lokalen Version den neuen Commit B des Servers hinzu

Synchronisierung lokal zu Server

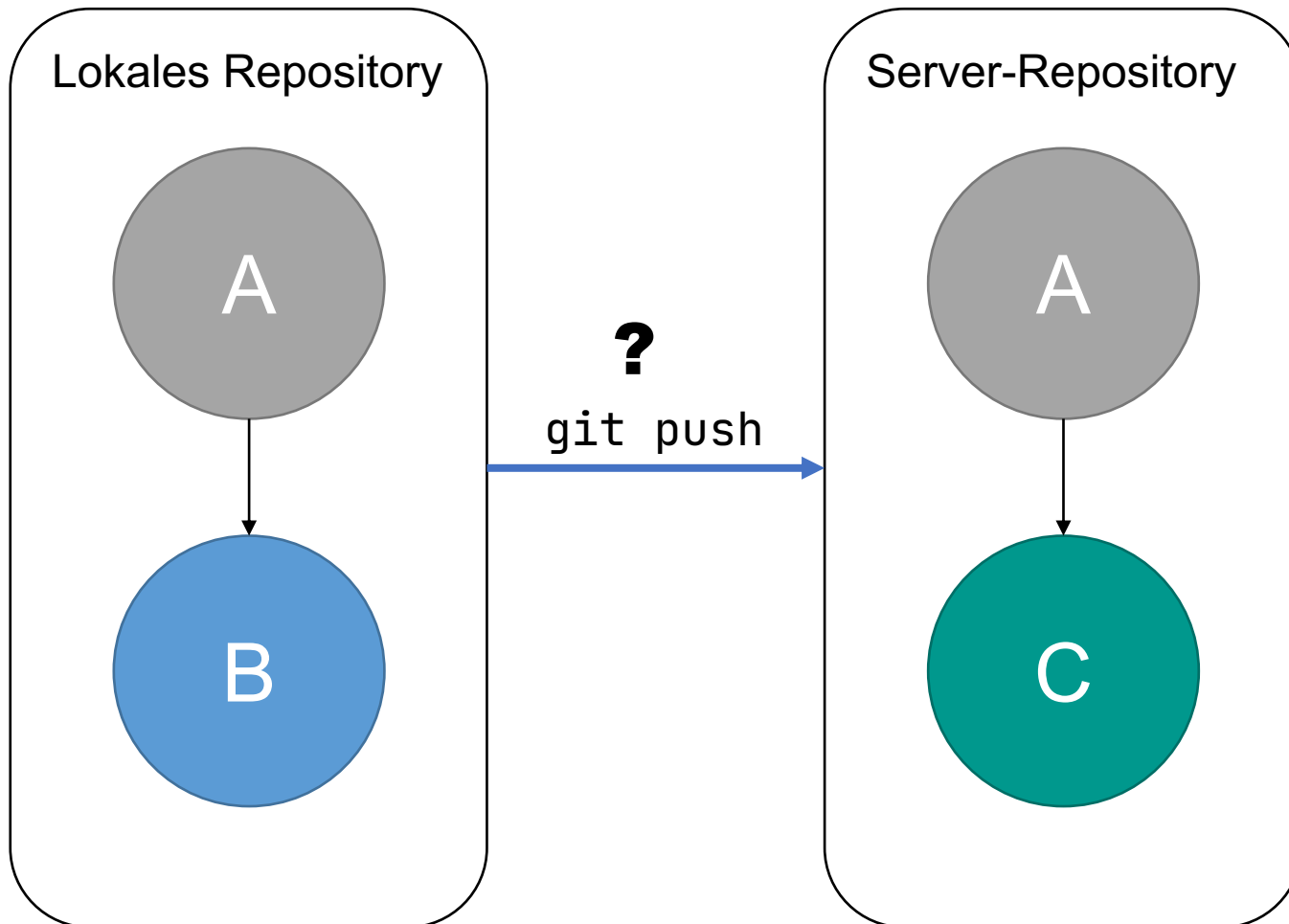


Synchronisierung lokal zu Server



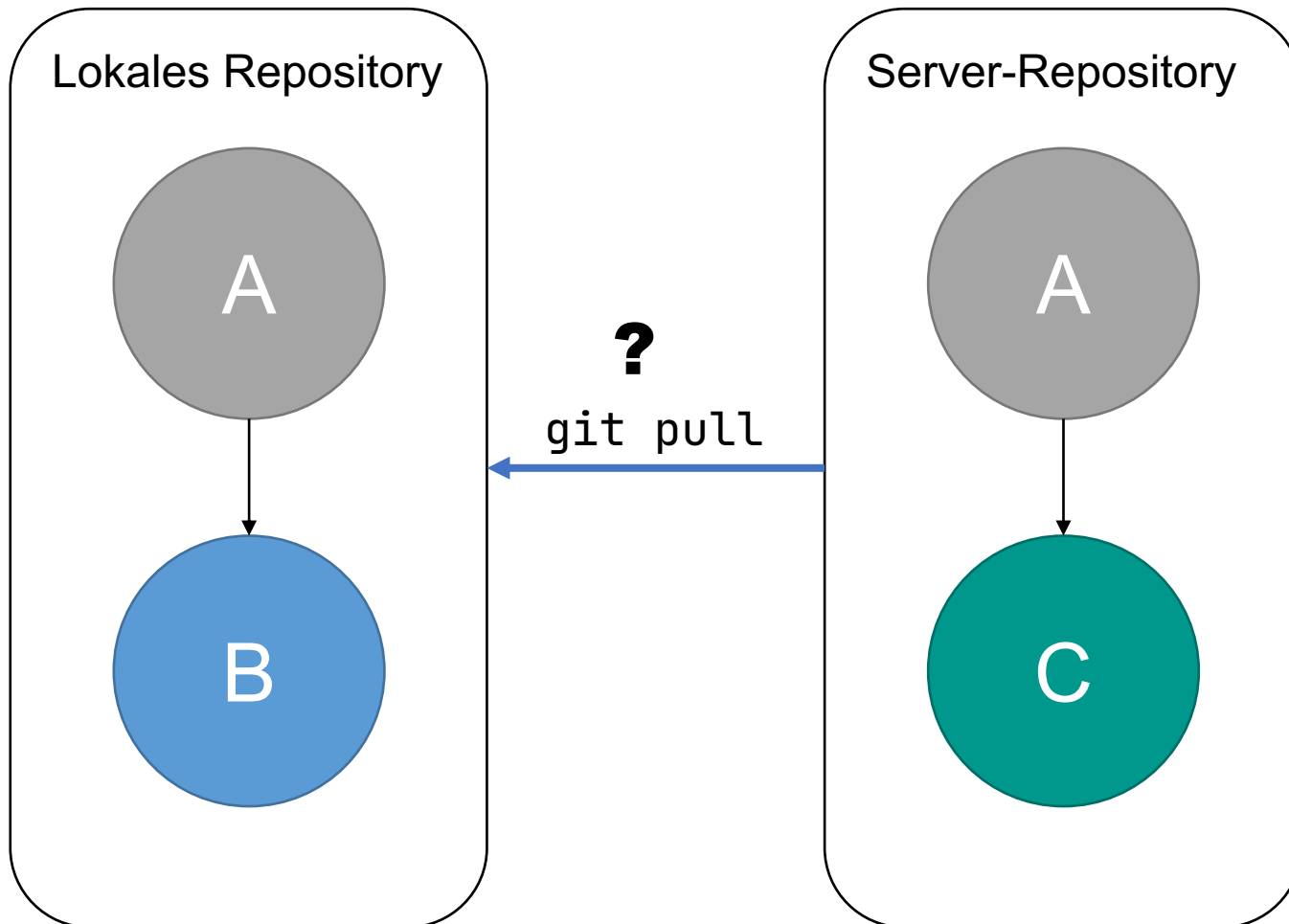
- `git push` fügt dem Server den neuen lokalen Commit B hinzu

Aufpassen, wenn lokal und Server auseinander laufen



- Ein Push ist hier **nicht möglich**, da Änderungen $A \rightarrow B$ die Änderungen $A \rightarrow C$ überschreiben würden.

Aufpassen, wenn lokal und Server auseinander laufen

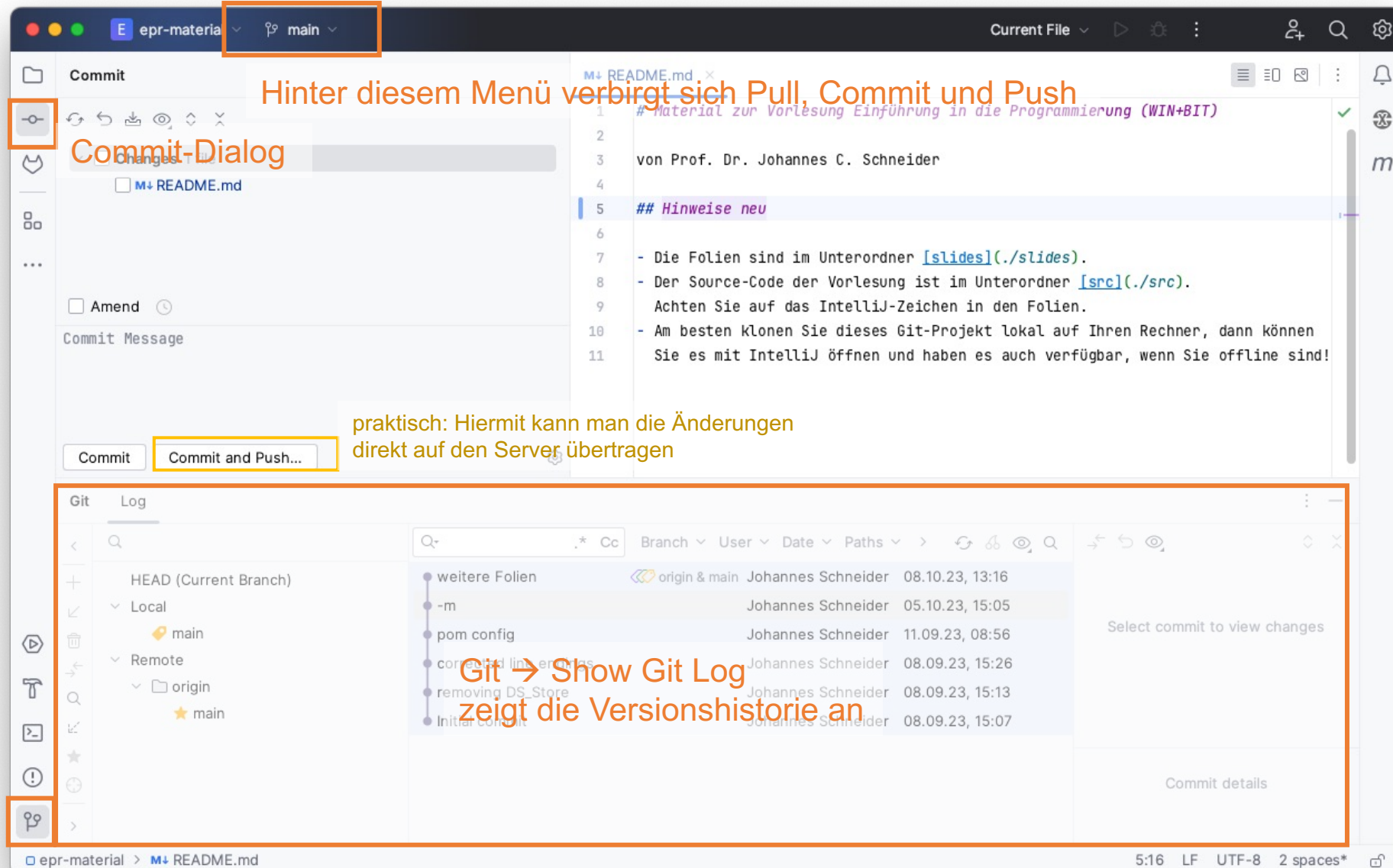


- Ein Pull hängt davon ab, wie man ihn ausführt

Weiteres

- [Ungewollte Änderungen rückgängig machen](#) (aus dem Git-Book) lesen!
- Es gibt viele Videos und Tutorials zum Thema Git!
- Beim Commit über die Kommandozeile die Message direkt mitangeben:
`git commit -m "Übungsaufgabe 3 bearbeitet"`
 - Ansonsten öffnet sich der Standard-Editor vi, der sehr gewöhnungsbedürftig ist
 - Man kann vi über die Tastenfolge *Doppelpunkt, c, q, Enter* beenden (dann wird der aktuelle Vorgang abgebrochen)
- Alternativ: Commit über IntelliJ (s. nächste Folie) – hier kann man auch direkt pushen.
- [Branches](#) (Verzweigungen) sind eine elegante Möglichkeit, mit mehreren Versionen eines Projektes gleichzeitig zu arbeiten → **nicht Teil dieser Vorlesung**

IntelliJ und Git



- gute Integration
- trotzdem am Anfang die Kommandozeile verwenden (View → Tool Windows → Terminal)